AeroMoE File Format — Session 1 Reference

Binary Layout (all sections 64 KB-aligned)

```
Offset          Section             Size
──────────────────────────────────────────────────────────────
0x0000_0000     FILE HEADER         512 B    (fixed)
0x0000_0200     DENSE INDEX         ≤64 KB   n_dense × 48 B entries
[aligned]       EXPERT INDEX        var      n_experts × 24 B entries
[aligned]       DENSE WEIGHTS       var      backbone tensors, tightly packed
[aligned]       EXPERT SLABS        var      one 64KB-aligned slab per (layer, expert)
```

File Header (512 bytes, little-endian)

Offset	Size	Field	Notes
0x000	8 B	magic	`"AEROMOE\0"`
0x008	4 B	version	`0x0001_0000` (major=1, minor=0)
0x00C	4 B	dtype_code	0=bf16 1=fp16 2=fp32 3=fp8_e4m3
0x010	4 B	hidden_size	
0x014	4 B	intermediate_size	Dense / shared-expert intermediate dim
0x018	4 B	moe_inter_size	Per-routed-expert intermediate dim
0x01C	4 B	num_layers	
0x020	4 B	n_heads	Attention heads
0x024	4 B	n_kv_heads	GQA key/value heads
0x028	4 B	vocab_size	
0x02C	4 B	max_seq_len	
0x030	4 B	num_experts	Routed experts per MoE layer
0x034	4 B	num_experts_per_tok	Top-k active per token
0x038	4 B	num_shared_experts	
0x03C	1 B	norm_topk_prob	1 = renormalize router probs after top-k
0x03D	1 B	tie_embeddings	1 = lm_head shares embed_tokens weights
0x03E	2 B	_pad	
0x040	4 B	rms_norm_eps	float32
0x044	4 B	rope_theta	float32

Offset	Size	Field	Notes
0x048	8 B	rope_type	char[8], e.g. `"yarn\0\0\0\0"`
0x050	8 B	dense_idx_offset	uint64, byte offset from file start
0x058	4 B	dense_idx_count	uint32
0x05C	4 B	_pad	
0x060	8 B	expert_idx_offset	uint64
0x068	4 B	expert_idx_count	uint32
0x06C	4 B	_pad	
0x070	8 B	dense_data_offset	uint64
0x078	8 B	expert_data_offset	uint64
0x080	384 B	reserved	zeros, future use

Dense Index Entry (48 bytes each)

```
struct DenseIndexEntry {
    uint64_t name_hash;     // FNV-1a 64-bit of tensor name string
    uint64_t offset;        // byte offset from file start
    uint64_t nbytes;        // stored byte count (after dtype conversion)
    uint32_t ndim;          // number of dimensions
    int32_t  shape[4];      // dimension sizes, unused dims = 0
    uint32_t dtype_code;    // same codes as header
    // total = 8+8+8+4+16+4 = 48 bytes
};
```

Expert Index Entry (24 bytes each)

```
struct ExpertIndexEntry {
    uint32_t layer_idx;     // transformer layer index (0-based)
    uint32_t expert_idx;    // expert index within layer (0-based)
    uint64_t offset;        // byte offset from file start to slab start
    uint64_t nbytes;        // slab byte count including alignment padding
    // total = 4+4+8+8 = 24 bytes
};
```

Expert Slab Internal Layout

Each slab stores three projections **tightly packed**, then zero-padded to the next 64 KB boundary:

```
[gate_proj.weight]  [up_proj.weight]  [down_proj.weight]  [zero padding → 64 KB bound-
ary]
```

For Qwen3.6-35B-A3B (bf16):
- `gate_proj.weight` shape: `[moe_intermediate_size, hidden_size]`
- `up_proj.weight` shape: `[moe_intermediate_size, hidden_size]`
- `down_proj.weight` shape: `[hidden_size, moe_intermediate_size]`

Usage

```
# Dry run — show layout plan, no file written
python aeromoe_convert.py \
    --model-dir ~/models/Qwen3.6-35B-A3B-Instruct \
    --output    ~/models/qwen36_35b_instruct.aeromoe \
    --dry-run

# Full conversion (keep source bf16 dtype)
python aeromoe_convert.py \
    --model-dir ~/models/Qwen3.6-35B-A3B-Instruct \
    --output    ~/models/qwen36_35b_instruct.aeromoe \
    --workers   8

# Convert + verify checksums on 32 random tensors
python aeromoe_convert.py \
    --model-dir ~/models/Qwen3.6-35B-A3B \
    --output    ~/models/qwen36_35b_base.aeromoe \
    --verify

# Force fp16 output (slightly smaller, marginal quality loss)
python aeromoe_convert.py \
    --model-dir ~/models/Qwen3.6-35B-A3B-Instruct \
    --output    ~/models/qwen36_35b_fp16.aeromoe \
    --dtype float16
```

Dependencies

```
pip install numpy safetensors tqdm
```

`safetensors` is only used for reference; the converter reads raw bytes directly via `pread()`-style seeks for maximum efficiency on large shards.

Sessions Roadmap

#	Component	Status
1	`safetensors → .aeromoe` converter	✅ Done
2	C++ engine core	Next
3	Metal kernels (Qwen3.6-specific)	Pending
4	Inference loop + model loader	Pending
5	Tool-calling layer	Pending